

Complete Guide to Git & GitHub in VS Code

Oct 1, 2026 · @Nicolas Catz

1. Introduction

This guide takes you from zero to a working Git workflow inside Visual Studio Code, using both the integrated terminal and the Source Control panel. Examples use Windows, with macOS and Linux notes where they differ.

Git is a version-control tool that runs on your computer. It records snapshots of your project (commits) so you can go back in time, work on features in parallel (branches) and collaborate.

GitHub and **GitLab** are websites that host Git repositories online (the "remote"). They add collaboration features: pull/merge requests, issues, code review, CI/CD.

	Git	GitHub	GitLab
What it is	Local version-control program	Online hosting service	Online hosting service (also self-hostable)
Where it runs	Your computer	github.com	gitlab.com or your own server
Collaboration unit	Branch	Pull request (PR)	Merge request (MR)
Cost	Free, open source	Free tier + paid plans	Free tier + paid plans

The key vocabulary:

- **Repository (repo):** a project folder tracked by Git, with its full history.
- **Commit:** a saved snapshot with a message.
- **Branch:** an independent line of work (the default is usually `main`).
- **Remote:** the online copy of your repo, conventionally named `origin`.
- **Clone:** download a remote repo to your computer.
- **Push / Pull:** send your commits to the remote / bring remote commits to your machine.
- **Staging area:** the "waiting room" where you choose which changes go into the next commit.

The basic loop you will repeat all day: **edit files** → **stage** → **commit** → **push**, and **pull** before you start working.

2. Installing and configuring Git

Windows

1. Download the installer from git-scm.com/download/win.
2. Run it. The defaults are fine; the options worth checking are: *Use Visual Studio Code as Git's default editor*, *Override the default branch name* set to `main`, and *Git Credential Manager* (enabled by default, it handles sign-in for you).
3. Alternatively, from PowerShell:

```
winget install --id Git.Git -e --source winget
```

macOS

```
xcode-select --install
```

or, with Homebrew: `brew install git`.

Linux (Debian / Ubuntu)

```
sudo apt update && sudo apt install git
```

Fedora: `sudo dnf install git`. Arch: `sudo pacman -S git`.

Verify the installation

Close and reopen VS Code (or your terminal) so the new PATH is loaded, then run:

```
git --version
```

You should see something like `git version 2.4x.x`.

First-time configuration

Git stamps every commit with a name and email. Use the same email as your GitHub/GitLab account so commits are linked to your profile.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

Check everything you set:

```
git config --global --list
```

Optional but useful on Windows, to avoid line-ending warnings:

```
git config --global core.autocrlf true
```

(On macOS/Linux use `input` instead of `true`.)

Privacy tip: GitHub lets you hide your real email. In *Settings* → *Emails* tick "Keep my email addresses private" and use the `ID+username@users.noreply.github.com` address it shows.

3. Creating an account and signing in

Create a GitHub account

1. Go to github.com and click **Sign up**.
2. Enter your email, a strong password and a username (it appears in your repo URLs, so choose something professional).
3. Verify your email address with the code GitHub sends you.
4. Enable two-factor authentication (**Settings** → **Password and authentication**). It is strongly recommended.

Students and teachers can get free Pro features through the [GitHub Education](https://education.github.com) program, using a university email.

Or create a GitLab account

Go to gitlab.com, click **Register**, and follow the same steps (email verification, 2FA). Everything below works the same way; GitLab-specific differences are in section 10.

Sign in from VS Code

There are three ways to authenticate. **Method A is the easiest and is enough for most people.**

Method A: browser sign-in with Git Credential Manager (HTTPS)

Installed with Git for Windows and macOS. The first time you `git push` or `git clone` a private repo over HTTPS, a browser window opens, you log in, and the token is stored securely. Nothing else to do.

Method B: sign in through VS Code

1. Click the **Accounts** icon (person silhouette) at the bottom-left of VS Code.
2. Choose **Sign in with GitHub to use GitHub Pull Requests...** (or install the *GitHub Pull Requests* extension first).
3. Your browser opens; click **Authorize Visual-Studio-Code**. You are now connected.

Method C: SSH keys (no repeated prompts, popular with developers)

Generate a key:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

Press Enter to accept the default location and choose an optional passphrase. Then print your public key:

```
type $env:USERPROFILE\.ssh\id_ed25519.pub
```

(macOS/Linux: `cat ~/.ssh/id_ed25519.pub`.) Copy the whole line.

On GitHub go to **Settings** → **SSH and GPG keys** → **New SSH key**, paste it, and save. Test the connection:

```
ssh -T git@github.com
```

You should see `Hi <username>! You've successfully authenticated`. Afterwards, use SSH URLs (`git@github.com:user/repo.git`) when cloning.

Never share your private key (the file without `.pub`), and never commit passwords, tokens or keys to a repository.

4. Getting to know VS Code's Git tools

VS Code has Git built in. Once Git is installed, everything below works with no extra setup.

The Source Control panel

Open it with the branch icon in the left Activity Bar, or press **Ctrl+Shift+G** (**Cmd+Shift+G** on macOS). It shows:

- **Message box** at the top: type your commit message here, then press **Ctrl+Enter** to commit.
- **Changes** list: every modified, new or deleted file. Hover a file to get icons: *open file*, *discard changes*, and **+** (stage).
- **Staged Changes** list: what will go into the next commit.
- **⋮ (More Actions)** menu: pull, push, fetch, branch, stash, and more.
- **Sync Changes** button: pushes and pulls in one click.

Clicking a file opens a side-by-side **diff** (left: previous version, right: your version).

Status bar and file badges

- Bottom-left: the **current branch name**. Click it to switch or create branches. The circular arrows icon next to it syncs with the remote, with counters like **1↓ 2↑** (1 commit to pull, 2 to push).
- File colors and letters in the Explorer: **U** = untracked (new), **M** = modified, **A** = added/staged, **D** = deleted, **C** = conflict.

The integrated terminal

Open it with **Ctrl+`** (backtick) or **Terminal → New Terminal**. It already starts in your project folder. Use it for any **git** command in this guide. The terminal and the Source Control panel work on the same repository, so you can mix them freely.

Windows tip: the default shell is PowerShell. If you prefer Git Bash, click the dropdown arrow next to **+** in the terminal panel and choose *Git Bash*.

Recommended extensions

Extension	What it adds
GitHub Pull Requests (by GitHub)	Create, review and merge PRs and issues without leaving VS Code
GitLens (by GitKraken)	Inline blame (who changed this line), rich history, comparisons
Git Graph (by mhutchie)	A visual graph of branches and commits

Extension	What it adds
GitLab Workflow (by GitLab)	Merge requests, issues and pipelines for GitLab users

Install from the Extensions view (`Ctrl+Shift+X`) by searching the name.

5. Creating a repository

There are three common starting points. Pick the one that matches your situation.

Option 1: start locally, then publish to GitHub (from VS Code)

Best when you already have a folder with code.

1. In VS Code: **File** → **Open Folder...** and select your project folder (create an empty one if needed).
2. Open the Source Control panel (`Ctrl+Shift+G`) and click **Initialize Repository**.
3. Create or edit some files, stage them with `+`, type a message (for example `Initial commit`) and click **Commit**.
4. Click **Publish Branch** (or **Publish to GitHub**). VS Code asks for the repository name and whether it should be *public* or *private*. Confirm, and it creates the repo on GitHub and pushes your code.

Terminal equivalent for the local part:

```
git init
git add .
git commit -m "Initial commit"
```

Option 2: create the empty repo on GitHub first, then connect

1. On GitHub click **+ → New repository** (top-right).
2. Fill in:
 - **Repository name** (no spaces; use `my-project`).
 - **Description** (optional).
 - **Public** (anyone can see it) or **Private** (only you and invited people).
 - Optionally tick *Add a README*, *.gitignore* template and *license*.
3. Click **Create repository**. GitHub shows a page with the URL and setup commands.
4. In your local project folder, run the commands GitHub displays. If the folder already contains code and you did **not** add a README on GitHub:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/my-project.git
git push -u origin main
```

The `-u` flag links your local `main` to `origin/main`, so later a plain `git push` or `git pull` is enough.

Check your remotes at any time:

```
git remote -v
```

Option 3: create on GitHub with a README, then clone

If you tick *Add a README* on GitHub, the repo is not empty and you simply **clone** it (next section) instead of using `git remote add`.

Using the GitHub CLI (optional)

The [GitHub CLI](#) creates repos in one command:

```
winget install --id GitHub.cli
gh auth login
gh repo create my-project --private --source=. --remote=origin --push
```

Initial files worth adding

- `README.md` : what the project is and how to run it.
- `.gitignore` : files Git must never track (see section 9).
- `LICENSE` : required if you want others to be allowed to reuse your code.

6. Cloning an existing repository

Cloning downloads the full repository (code and history) and automatically sets `origin` to the source URL.

Get the URL

On the repository page on GitHub, click the green **Code** button and copy the **HTTPS** URL (`https://github.com/user/repo.git`) or the **SSH** URL (`git@github.com:user/repo.git`) if you configured SSH.

Clone from the terminal

First move to the folder where you keep your projects, then clone:

```
cd ~/Documents/projects
git clone https://github.com/user/repo.git
```

This creates a `repo` folder. To choose another folder name: `git clone <url> my-folder`. Then open it in VS Code:

```
cd repo
code .
```

Clone from VS Code

1. Open the Command Palette with `Ctrl+Shift+P` (`Cmd+Shift+P` on macOS).
2. Type **Git: Clone** and press Enter.
3. Paste the repository URL, or choose **Clone from GitHub** to pick from a list of your repos (requires being signed in).
4. Select the parent folder where the project folder will be created.
5. When VS Code asks *Would you like to open the cloned repository?*, click **Open**.

Shortcut: on the Welcome page or an empty Source Control panel, click **Clone Repository**.

Useful clone variations

Goal	Command
Clone a specific branch	<code>git clone -b dev <url></code>
Faster clone, latest snapshot only	<code>git clone --depth 1 <url></code>
Clone including submodules	<code>git clone --recurse-submodules <url></code>

Your own copy of someone else's project: fork

You cannot push to repositories you do not own. To contribute to someone else's project, click **Fork** on GitHub (it copies the repo into your account), clone *your fork*, make changes, push, and then open a pull request to the original project.

7. The daily workflow: status, commit, push, pull

A typical session looks like this:

1. **Pull** the latest changes.
2. **Edit** your files.
3. **Review** what changed (`git status` , `git diff`).
4. **Stage** the changes you want to save.
5. **Commit** them with a clear message.
6. **Push** to the remote.

Each step is shown below in the terminal and in the Source Control panel.

Pull: get the latest changes

Do this *before* you start working, to avoid conflicts later.

```
git pull
```

In VS Code: `...` → **Pull**, or the **Sync Changes** button (it pulls then pushes).

`git fetch` only downloads remote changes without merging them, so you can inspect them first. In VS Code: `...` → **Fetch**.

```
git fetch
git log HEAD..origin/main --oneline
```

Status and diff: see what changed

```
git status          # which files changed, which are staged
git diff            # line-by-line changes not yet staged
git diff --staged   # changes already staged
```

In VS Code, all of this is the Source Control panel: click a file to see its diff.

Stage: choose what goes in the commit

```
git add file.txt    # one file
git add src/        # a folder
git add .           # everything in the current folder
```

In VS Code: hover a file and click **+**, or hover the **Changes** header and click **+** to stage everything. You can even stage single lines: open the diff, select lines, right-click, **Stage Selected Ranges**.

To unstage: `git restore --staged file.txt` , or click **-** in the panel.

Commit: save a snapshot

```
git commit -m "Add login form validation"
```

In VS Code: type the message in the box and press **Ctrl+Enter** (or click **Commit**). If nothing is staged, VS Code offers to stage everything for you.

What makes a good commit message:

- Short summary in the imperative mood, under about 50 characters: **Fix crash when file is empty**.
- One logical change per commit, so history stays readable and easy to undo.
- Avoid **update**, **fix stuff** and **asdf**.

Fix the last commit message (before pushing): `git commit --amend -m "New message"`.

Push: send your commits to GitHub

```
git push
```

The first push of a new branch needs: `git push -u origin my-branch`. In VS Code: **...** → **Push, Sync Changes**, or **Publish Branch** for a brand-new branch.

If push is rejected with *"Updates were rejected because the remote contains work that you do not have locally"*, run `git pull` first, resolve any conflict, then push again.

See the history

```
git log --oneline --graph --decorate -15
```

In VS Code: the **Timeline** view at the bottom of the Explorer shows the history of the open file; GitLens and Git Graph show the whole repository.

Terminal vs. Source Control panel at a glance

Action	Terminal	Source Control panel
Get remote changes	<code>git pull</code>	... → Pull, or Sync Changes
See changes	<code>git status</code> , <code>git diff</code>	Changes list, click a file
Stage	<code>git add <file></code>	+ next to the file
Unstage	<code>git restore --staged <file></code>	- next to the file
Commit	<code>git commit -m "msg"</code>	Message box + Ctrl+Enter

Action	Terminal	Source Control panel
Push	<code>git push</code>	... → Push, or Sync Changes
Discard changes in a file	<code>git restore <file></code>	Discard icon (curved arrow)

8. Branches, pull requests and merge conflicts

A branch lets you work on a feature or fix without touching `main`. When it is ready, you merge it back, ideally through a pull request so others can review it.

Create and switch branches

```
git branch          # list local branches
git switch -c feature/login  # create AND switch to a new branch
git switch main      # go back to main
git switch feature/login  # switch to an existing branch
```

In VS Code: click the branch name in the bottom-left status bar. Choose **Create new branch...** or pick an existing branch from the list. The same options exist under **... → Branch**.

Push the new branch for the first time:

```
git push -u origin feature/login
```

The recommended team workflow (GitHub flow)

1. `git switch main` and `git pull` to start from the latest code.
2. `git switch -c feature/short-description`.
3. Commit your work in small steps; push the branch regularly.
4. Open a **pull request** on GitHub from your branch into `main`.
5. A teammate reviews it; you fix comments with extra commits.
6. Merge the pull request, then delete the branch.
7. Locally: `git switch main`, `git pull`, and `git branch -d feature/short-description`.

Open a pull request

On GitHub: after pushing a branch, a yellow banner *Compare & pull request* appears on the repo page. Click it, write a title and description, then **Create pull request**.

In VS Code: with the *GitHub Pull Requests* extension, click **Create Pull Request** in the Source Control panel after publishing your branch. Reviewing, commenting and merging also work from the extension's sidebar.

Choose the merge style on GitHub's merge button: **Create a merge commit** (keeps full history), **Squash and merge** (one tidy commit; popular default) or **Rebase and merge** (linear history).

Merge locally

```
git switch main
git pull
git merge feature/login
git push
```

In VS Code: **⋮ → Branch → Merge Branch...**, choose the branch to merge into the current one.

Keep your branch up to date

If `main` moved on while you worked, bring it into your branch:

```
git switch feature/login
git fetch
git merge origin/main
```

Delete a branch

```
git branch -d feature/login          # local, safe (refuses if unmerged)
git push origin --delete feature/login  # remote
```

Resolving merge conflicts

A conflict happens when two branches changed the same lines. Git stops the merge and marks the files. In VS Code they appear under **Merge Changes** with a **C** badge. Inside a file you will see:

```
<<<<<< HEAD
your version of the line
=====
the incoming version of the line
>>>>>> origin/main
```

VS Code shows clickable options above the markers:

- **Accept Current Change:** keep your version.
- **Accept Incoming Change:** keep theirs.
- **Accept Both Changes:** keep both.
- **Compare Changes:** side-by-side view. You can also click **Resolve in Merge Editor** for a three-way editor.

After resolving every conflict in a file, save it and stage it. When all files are resolved, finish the merge:

```
git add .  
git commit      # opens the prefilled merge message; save and close it
```

To cancel a merge and go back to how things were: `git merge --abort`.

Reduce conflicts: pull often, keep branches short-lived, and avoid reformatting whole files you are not working on.

9. Undoing mistakes, .gitignore, stash and more

Undoing things

Situation	Command	Safe on shared history?
Discard unstaged edits in a file	<code>git restore file.txt</code>	Yes (but the edits are lost for good)
Unstage a file	<code>git restore --staged file.txt</code>	Yes
Fix the last commit message or add a forgotten file	<code>git add file</code> then <code>git commit --amend</code>	Only if not pushed yet
Undo the last commit, keep the changes staged	<code>git reset --soft HEAD~1</code>	Only if not pushed yet
Undo the last commit, keep the changes unstaged	<code>git reset HEAD~1</code>	Only if not pushed yet
Undo a commit that was already pushed	<code>git revert <commit-id></code>	Yes (adds a new commit that cancels the old one)
Throw away everything and match the remote	<code>git fetch</code> then <code>git reset --hard origin/main</code>	Destructive: local work is lost

In VS Code: **⋮ → Commit → Undo Last Commit**, and the discard icon next to a file. Use `git revert` rather than `reset` for anything others may already have.

Committed work is almost never truly lost. `git reflog` lists where HEAD has been, and you can recover a "lost" commit with `git switch -c rescue <commit-id>`.

.gitignore: files Git should not track

Create a file named `.gitignore` at the root of the repo, one pattern per line:

```
# dependencies and build output
node_modules/
dist/
__pycache__/
*.pyc

# secrets and local settings
.env
*.key

# OS and editor files
.DS_Store
Thumbs.db
.vscode/
```

GitHub offers ready-made templates at [github/gitignore](https://github.com/gitignore). Important: `.gitignore` only affects files that are not yet tracked. To stop tracking a file already committed:

```
git rm --cached secrets.env
```

If you accidentally pushed a password or token, **revoke/rotate it immediately**; deleting it from a later commit does not remove it from history.

Stash: park unfinished work

You need to switch branches but are not ready to commit:

```
git stash push -m "half-done login form"
git switch other-branch
# ...do something...
git switch -
git stash pop
```

Other commands: `git stash list`, `git stash apply`, `git stash drop`. In VS Code: ...
→ **Stash** → **Stash**, and **Pop Stash** to restore.

Tags: mark a release

```
git tag -a v1.0.0 -m "First release"
git push origin v1.0.0
```

Inspect history and blame

```
git log --oneline -- path/to/file    # history of one file
git show <commit-id>                # what a commit changed
git blame file.txt                  # who last changed each line
```

Other useful commands

```
git remote set-url origin <new-url> # change the remote URL
git commit -am "msg"                 # stage tracked files + commit in one
go
git config --global alias.st status  # create a shortcut: git st
```

10. GitLab, troubleshooting, best practices and cheat sheet

Using GitLab instead of GitHub

The Git commands and the VS Code Source Control panel are identical. Only the website differs:

GitHub	GitLab
Pull request	Merge request (MR)
New repository	New project (+ → New project/repository)
Actions (CI/CD)	Pipelines (<code>.gitlab-ci.yml</code>)
Settings → SSH and GPG keys	Preferences → SSH Keys
Remote URL <code>github.com/user/repo</code>	<code>gitlab.com/user/repo</code> (or your organization's server)

To clone: copy the URL from the blue **Code** button and use `git clone` or **Git: Clone** in VS Code. For authentication, HTTPS needs a **Personal Access Token** in place of a password (**Preferences → Access Tokens**, scope `read_repository` and `write_repository`), or use SSH keys. Install the *GitLab Workflow* extension to manage merge requests from VS Code. Universities and companies often run their own GitLab server; just use its URL.

You can add both as remotes on one project:

```
git remote add github https://github.com/user/repo.git
git remote add gitlab https://gitlab.com/user/repo.git
git push github main
git push gitlab main
```

Troubleshooting

Problem	Cause and fix
'git' is not recognized	Git is not installed or not in PATH. Reinstall, then restart VS Code.
Author identity unknown / Please tell me who you are	Set <code>user.name</code> and <code>user.email</code> (section 2).
Authentication failed / Permission denied	Password sign-in is no longer accepted. Use browser sign-in, a personal access token, or SSH. On Windows, clear old entries in <i>Credential Manager</i> → <i>Windows Credentials</i> and try again.
rejected ... fetch first / non-fast-forward	The remote has commits you lack. <code>git pull</code> , resolve conflicts, <code>git push</code> .
fatal: not a git repository	You are in the wrong folder, or never ran <code>git init</code> . Check with <code>pwd</code> and <code>git status</code> .
fatal: refusing to merge unrelated histories	Two repos with different starts (often a README created on GitHub). <code>git pull origin main --allow-unrelated-histories</code> .
There is no tracking information for the current branch	<code>git push -u origin <branch></code> or <code>git branch --set-upstream-to=origin/<branch></code> .
VS Code shows no Git panel	Folder is not a repo (click Initialize Repository), or the <code>.git</code> folder is in a parent directory, or <code>git.enabled</code> is off in settings.
detected dubious ownership	<code>git config --global --add safe.directory <path></code> for a folder owned by another account.
File over 100 MB rejected by GitHub	Use Git LFS , or keep large files out of the repo.

Problem	Cause and fix
Detached HEAD	You checked out a commit or tag. <code>git switch -c new-branch</code> to keep your work, or <code>git switch main</code> to leave.

Best practices

- **Commit small and often**, one logical change at a time, with meaningful messages.
- **Pull before you start** and before you push.
- **Never work directly on `main`** in a team; use branches and pull requests.
- **Never commit secrets** (passwords, API keys, `.env` files). Add them to `.gitignore` first.
- **Do not commit generated files** (build output, dependencies, caches).
- **Write a useful README** and keep it up to date.
- **Do not rewrite shared history**: avoid `push --force` on branches others use. If you must, prefer `git push --force-with-lease`.
- **Protect `main`** on GitHub (*Settings* → *Branches*) to require pull requests and reviews.
- **Back up by pushing**: an unpushed commit exists only on your machine.

Cheat sheet

```
# Setup (once)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Start
git init                # new repo in current folder
git clone <url>          # copy a remote repo

# Daily loop
git pull                # get latest
git status              # what changed?
git add .               # stage everything
git commit -m "message" # save snapshot
git push                # upload

# Branches
git branch              # list
git switch -c name      # create + switch
git switch name         # switch
git merge name          # merge into current branch
```

```

git branch -d name                # delete

# Inspect
git log --oneline --graph         # history
git diff                         # unstaged changes
git remote -v                    # remotes

# Undo
git restore file                  # discard local edits
git restore --staged file         # unstage
git revert <commit>               # undo a pushed commit safely
git stash / git stash pop         # park / restore work

```

VS Code shortcuts

Action	Windows / Linux	macOS
Source Control panel	Ctrl+Shift+G	Cmd+Shift+G
Command Palette (type "Git:")	Ctrl+Shift+P	Cmd+Shift+P
Integrated terminal	Ctrl+`	Ctrl+`
Commit (in message box)	Ctrl+Enter	Cmd+Enter
Extensions	Ctrl+Shift+X	Cmd+Shift+X

Where to go next

- [Pro Git book](#) (free, official)
- [GitHub Docs](#) and [GitLab Docs](#)
- [VS Code: Source Control overview](#)
- Practice safely at learngitbranching.js.org